

Software Testing

Spectrum-based Fault Localization

Universidade do Minho

2025/2026

1 Fault Localization

Software testing does not establish the correctness of the program under test. However, a failing test case provides evidence of the presence of a fault. When a fault is revealed by the test suite, it is essential to perform fault localization, that is, to identify the location of the fault within the source code of the program under test.

One widely used approach is *spectrum-based fault localization* (SFL), which leverages execution information collected from running the test suite. In this setting, each program element (e.g., line, statement, or branch) is associated with a spectrum describing whether it was executed by *passing* and *failing* test cases. By applying a similarity coefficient (such as the *Jaccard* or *Ochiai* coefficient) to this information, program elements can be ranked according to their likelihood of being faulty, thereby guiding developers to the most suspicious parts of the code.

2 Exercise

Implement the spectrum-based fault localization technique to identify the fault in the following C function, which is intended to return the greatest of its three arguments.

```
1  int greatest(int a, int b, int c)
2  { int m;
3      if (! (a > b))
4          { if (b > c)
5              m = b;
6          else m = c; }
7      else
8          { if (a > c)
9              m = a;
10         else m = b; }
11     return m;
12 }
```

Rewrite this faulty function in your preferred programming language and answer the following questions.

1. Modify this function so that it not only returns the greatest number, but also produces an execution trace of the program components that were evaluated.

Suggestion: Use program lines as components and represent the trace as a list of integers, where the value at position n is 1 if the line n was executed and 0 otherwise.

2. Execute this function using a suitable test suite, including both passing and failing test cases.

Suggestion: Define a data type `MatrixSFL` that, for each test execution, records:

- the execution trace (i.e., the list of integers computed as described above), and
- the result of the test (pass or fail).

3. Implement the function `jaccard :: MatrixSFL -> [Float]`, which takes as input the SFL matrix and computes the suspiciousness score of each program component (i.e., each line) using the Jaccard coefficient.

The Jaccard coefficient J is computed as follows:

For each component i :

$$J(i) = \frac{a}{a + b + c}$$

where:

- a = number of test cases that fail and execute component i
- b = number of test cases that pass and execute component i
- c = number of test cases that fail that **do not** execute component i

4. Implement the function `ochiai :: SFLMatrix -> [Float]` that computes the suspiciousness score using the Ochiai coefficient.

The Ochiai coefficient O for each program's component i is computed as follows

For each component i :

$$O(i) = \frac{a}{\sqrt{(a + b)(a + c)}}$$

where:

- a = number of test cases that fail and execute component i
- b = number of test cases that pass and execute component i
- c = number of test cases that fail that **do not** execute component i

5. Analyse the results produced by the two coefficients running the same test suite.
 - (a) Which components/lines are identified as most suspicious?
 - (b) Which coefficient provides more accurate fault localization results?
6. Inject a second fault in a different component/line of function **greatest** and re-execute your fault localization framework.
 - (a) Which components/lines are identified as most suspicious?
 - (b) Which coefficient provides more accurate fault localization results?
7. Correct the faulty component in the **greatest** function and execute the test suite again using your fault localization framework.